



中山大學
SUN YAT-SEN UNIVERSITY

信息安全技术期末考查

Watermark for LLM 三种算法对比实验

姓 名 _____ 待填写

学 号 _____ 待填写

学 院 _____ 待填写

专 业 _____ 待填写

2026 年 7 月 9 日

摘要

大语言模型生成文本的自动化能力正在快速提升，随之而来的学术诚信、虚假信息生成和合成数据污染等问题，使文本水印成为信息安全领域的重要研究方向。本文围绕三种代表性 LLM 水印方案展开复现与对比：KGW 通过伪随机绿名单和 logit 偏置在生成阶段嵌入统计信号；SWEET 针对代码等低熵场景，只在高熵 token 上嵌入和检测水印；EWD 则在检测阶段引入熵加权统计量，为高熵 token 分配更高权重。实验实现了从数据读取、模型生成、水印嵌入、水印检测、perplexity 评估到结果作图的完整链路。正式云端实验使用 OPT-1.3B 生成、OPT-6.7B 计算 PPL，在 C4 RealNewsLike 500 条新闻文本和 HumanEval 低熵代码任务上评估三种方法。结果表明，三种方法均能有效区分 human 与 watermarked 文本；SWEET 在保持高检测性能的同时带来更低 PPL 增量；EWD 适合作为 KGW 生成文本的增强检测器，但本身不是独立生成水印方案。

1 背景和动机

生成式大语言模型已经能够完成新闻写作、课程作业、代码生成和问答等任务。模型输出越接近人类文本，越难通过人工方式判断来源，这会带来三个典型风险。第一，恶意用户可以批量生成虚假新闻、评论或社交媒体内容，增加内容审核难度。第二，学生或程序员可能提交模型生成文本或代码，造成学术诚信与版权归属争议。第三，如果未来训练数据中混入大量机器生成内容，模型训练和评测本身也可能被污染。因此，能够在文本生成阶段主动嵌入、在事后自动检测的水印机制，是一类具有现实意义的信息安全技术。

与事后分类器不同，水印方法并不是在文本生成后再尝试识别写作风格，而是在生成过程中主动改变采样分布，使输出文本携带可统计检验的隐藏信号。这种思路的优势是检测结论可以用显式统计量解释，例如绿名单 token 的数量是否显著高于随机文本；不足之处是水印必须参与生成流程，且水印强度过高时可能损害文本质量。本文关注的 KGW、SWEET 和 EWD 都属于这一类围绕 token 采样分布设计的水印方法，因此实验重点放在“水印强度、检测准确性、文本质量损失”三者之间的权衡。

传统数字水印常见于图像、音频和视频，其核心是在人类难以察觉的载体空间中嵌入可检测信号。LLM 文本水印的困难在于文本是离散 token 序列，任何 token 修改都可能改变语义、语法或代码功能。KGW [1] 提出了一个简单有效的方向：不直接修改已生成文本，而是在模型采样时轻微提高部分 token 的概率，让最终文本在统计上包含更多“绿名单 token”。SWEET [2] 观察到代码生成等任务的 token 熵较低，强行偏置低熵 token 会破坏功能，偏置太弱又难以检测，于是提出只处理高熵 token。EWD [3] 则认为检测阶段也应充分利用熵信息，不能把所有 token 一视同仁。三者形成从“基础生成水印”到“选择性生成水印”再到“熵加权检测”的发展脉络。

2 方法原理

2.1 KGW 水印

KGW 的基本思想是：给定前文 token 和密钥，通过哈希函数确定一个伪随机种子，再把词表划分为绿名单 G 和红名单 R 。绿名单比例记为 γ 。在生成第 t 个 token 时，模型先输出原始 logit 向量，然后对所有绿名单 token 的 logit 加上偏置 δ ，再进行采样。若原始分布较高熵，绿名单 token 的采样概率会明显高于 γ ；若文本不是由该水印生成，则 token 落入绿名单的概率近似为 γ 。

这里的关键不是固定某些词必须出现，而是轻微改变候选 token 的相对概率。对高熵位置而言，模型本来就有多个合理候选，给绿名单 token 加上 δ 往往不会明显破坏语义，却会累积出可检测的统计偏差。对低熵位置而言，模型几乎只有一个合理 token，例如代码缩进、括号、固定关键字或常见 API 片段，此时 logit 偏置要么不起作用，要么会把输出推向不自然甚至错误的 token。因此 KGW 的简单性也带来一个问题：它对所有位置一视同仁，难以区分“适合嵌入水印的位置”和“最好不要干预的位置”。

检测时，检测器根据同一密钥和前文 token 复原每个位置的绿名单，统计待检测文本中绿名单 token 数 $|s|_G$ 。对于长度为 T 的文本，KGW 使用一侧 z 检验：

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}}.$$

当 z 超过阈值时，判定文本含有水印。论文中常用 $z > 4$ 作为较严格阈值，对应较低误报概率。

KGW 检测端的另一个优势是轻量：只要知道密钥、绿名单生成规则和待检测文本，就能复现每个位置的绿名单，不需要重新运行原始生成模型。这一点适合平台侧或第三方审核场景。但如果要分析 token 熵、PPL 或与 SWEET/EWD 做同一框架下的比较，实验中仍需要加载语言模型来获得 logits 或计算质量指标。

2.2 SWEET 水印

KGW 在自然语言新闻生成等高熵场景中效果较好，但在代码生成中会遇到两类问题：低熵 token 往往由语法或上下文强约束决定，强行提高绿名单概率可能破坏代码正确性；同时，低熵 token 很难被水印偏置改变，继续把它们计入检测统计量会稀释 z -score。SWEET 因此引入熵阈值 τ 。在生成阶段，只有当当前位置模型分布熵 $H_t > \tau$ 时才执行 KGW 式绿名单 logit 偏置；在检测阶段，也只统计熵高于阈值的 token。

换言之，SWEET 把 KGW 的“全位置加偏置”改成了“选择性加偏置”。如果某个位置的分布非常尖锐，说明模型对下一个 token 很确定，SWEET 会跳过该位置，避免为了水印而破坏代码结构或固定表达。如果某个位置分布较平坦，说明有多个候选都较自然，此时再施加绿名单偏置，既更容易产生绿名单 token，也更不容易造成明显质量

下降。这个设计使 SWEET 同时改动了生成端和检测端：生成端少干预低熵 token，检测端也不让低熵 token 稀释统计量。

设高熵 token 数为 N^h ，其中绿名单 token 数为 N_G^h ，SWEET 的检测统计量为：

$$z = \frac{N_G^h - \gamma N^h}{\sqrt{N^h \gamma (1 - \gamma)}}.$$

这相当于过滤掉难以被水印影响的低熵位置，使检测更集中于水印真正可能改变采样结果的区域。

熵阈值 τ 的选择决定了 SWEET 的取舍。阈值过低时，SWEET 接近 KGW，质量保护有限；阈值过高时，能保护更多低熵位置，但可计分 token 数减少，检测统计量可能不稳定。本文没有额外搜索阈值，而是保持统一配置，让 KGW、SWEET 和 EWD 在相同实验框架下比较，这样可以突出三种方法机制本身的差异。

2.3 EWD 检测

EWD 的核心贡献不在于提出新的生成器，而在于改造检测统计量。KGW 给所有 token 相同权重，SWEET 给高熵 token 权重 1、低熵 token 权重 0。EWD 认为 token 对检测结论的影响应与熵连续相关：高熵 token 更容易被水印偏置改变，因此应在检测中占更高权重；低熵 token 即使不在绿名单，也不应强烈削弱水印判断。

本文实现采用 EWD 论文中的线性权重形式。对第 i 个被检测 token，根据模型分布计算 spike entropy SE_i ，再令权重 $W_i = \max(SE_i - C_0, 0)$ 。检测统计量为：

$$z' = \frac{|s|_G - \gamma \sum_i W_i}{\sqrt{\gamma(1 - \gamma) \sum_i W_i^2}},$$

其中 $|s|_G$ 表示绿名单 token 权重和。EWD 可以用于 KGW 或 SWEET 生成的文本；本实验为了隔离变量，默认让 EWD 复用 KGW 生成文本，只替换检测器。

这种设置有助于避免把“生成质量变化”和“检测统计量变化”混在一起。由于 EWD 与 KGW 使用同一批生成文本，它们的 PPL 完全相同；如果 EWD 的 AUROC、TPR 或 z-score 更高，差异就来自检测端的熵加权，而不是来自生成文本本身更容易检测。EWD 的代价是检测时需要模型 logits 来计算 token 熵，因此部署成本高于只复现绿名单的 KGW 检测器。

3 实验设计

3.1 模型和数据

正式实验遵循统一模型原则。生成模型配置为 facebook/opt-1.3b，PPL/oracle 模型配置为 facebook/opt-6.7b，OPT 模型来自 Open Pre-trained Transformer 系列 [4]。检测和生成均使用同一密钥、同一绿名单比例 $\gamma = 0.25$ 、同一 logit 偏置 $\delta = 2.0$ 。

选择统一模型而不是分别复现每篇论文的全部模型设置，是为了控制变量。KGW 原论文主要关注通用文本生成，SWEET 原论文使用代码模型并评估 $\text{pass}@1$ ，EWD 论文同时比较低熵代码和高熵文本。若完全照搬三篇论文的模型与数据，实验差异会同时来自算法、模型能力、数据分布和评测指标，难以判断某个指标变化究竟来自哪一部分。因此本文将三种算法放入同一实现框架中，对生成模型、检测密钥、采样参数和 PPL 计算方式进行统一，只比较水印生成器和检测器的差异。

主实验使用 C4 的 RealNewsLike 子集 [5]，共 500 条。每条样本文本被切分为 prompt 和 human completion，prompt 输入生成模型，生成长度设为 200 tokens；同一条 prompt 分别生成无水印文本、KGW 水印文本和 SWEET 水印文本，EWD 对 KGW 水印文本进行加权检测。

补充实验改用 HumanEval 低熵代码任务。HumanEval 每条样本包含函数签名、自然语言说明、示例和参考实现，更接近 SWEET 论文中的真实代码生成评测。项目早期使用过 200 条合成结构化 Python 样本，但该数据集区分度过强，三种方法的检测指标容易同时饱和，因此不再作为低熵对比的主要数据集。

在数据组织上，每条样本都包含 prompt 和 human_completion。human completion 作为负样本，用来估计误报；模型生成的水印 completion 作为正样本，用来估计检出率。HumanEval 的 prompt 通常已经包含函数签名、文档字符串和示例，这会让后续代码补全呈现更强的结构约束，因此比普通新闻文本更接近低熵场景。

3.2 评价指标

本文使用以下指标对比三种算法：

- z-score：检测统计量，越高表示越可能含有水印。
- AUROC：将 human completion 作为负样本、水印生成文本作为正样本，衡量检测分数排序能力。
- TPR@5%FPR：在误报率不超过 5% 时的检测召回率。
- $z > 4$ 检出率和误报率：对应 KGW 论文中常用的严格检测阈值。
- PPL：用 OPT-6.7B oracle 模型计算生成 completion 的困惑度，反映生成质量变化。
- 可计分 token 数和绿名单比例：用于解释水印嵌入强度和检测分数。

这些指标对应不同问题。AUROC 更关注排序能力，即水印文本是否整体排在人类文本前面；TPR@5%FPR 更接近实际审核场景，因为误报过高会影响正常用户； $z > 4$ 检出率则对应 KGW 论文中常用的固定阈值，便于解释统计显著性。PPL 不直接等于人类主观质量或代码正确率，但在统一 oracle 模型下可以反映生成分布是否被水印过度

扭曲。对于 HumanEval, 本文没有执行测试用例计算 pass@1, 因此低熵实验的质量结论应理解为 PPL 层面的质量代理, 而不是完整代码功能正确性结论。

3.3 代码实现

项目代码实现了完整实验链路。水印模块包含 KGW 和 SWEET 的生成处理器, 可直接接入 HuggingFace 生成流程; 同一模块还实现 KGW、SWEET 和 EWD 三类检测器。为保证检测和生成跨 CPU/GPU 一致, 绿名单伪随机排列固定在 CPU 上生成, 再搬运到目标设备。实验入口负责读取配置、加载模型、生成文本、计算检测分数和 PPL, 并输出 CSV/JSONL; 作图脚本根据结果绘制 z-score 分布、ROC 曲线和质量-可检测性权衡图。

实验输出分为三类文件: `generations.jsonl` 保存每个样本的 prompt、生成文本、PPL 和正样本 z-score, 便于追查具体失败案例; `scores.csv` 保存 human 与 watermarked 两类文本的逐样本检测分数; `metrics.csv` 汇总 AUROC、TPR@5%FPR、平均 z-score 和平均 PPL 等指标。EWD 在配置中通过 `reuse_generation_from: KGW` 复用 KGW 生成文本, 因此同一批文本可以分别由 KGW 检测器和 EWD 检测器打分。

4 实验结果与分析

4.1 C4 RealNewsLike 主实验

表 1 给出 C4 RealNewsLike 500 条主实验结果。三种方法都能有效区分 human 与 watermarked 文本, AUROC 均高于 0.995。在 $z > 4$ 的固定阈值下, 三种方法的误报率均为 0, 检出率分别为 92.2%、94.6% 和 94.0%。

C4 新闻文本整体熵较高, 模型在许多位置存在多个自然候选, 因此三种方法都能形成明显的绿名单偏置。KGW 的平均水印 z-score 为 8.839, 已经远高于严格阈值; SWEET 在只选择高熵 token 的情况下仍达到 9.716, 说明跳过低熵位置并没有削弱总体检测, 反而减少了一些无效或有害的干预; EWD 的 9.669 则说明熵加权检测在高熵文本中也能保持较强排序能力。

表 1: C4 RealNewsLike 500 条主实验结果

算法	水印 z	human z	AUROC	TPR@5%FPR	$z > 4$ 检出	$z > 4$ 误报	平均 PPL	PPL 增量
KGW	8.839	-0.134	0.9955	0.990	92.2%	0%	21.388	+7.986
SWEET	9.716	-0.057	0.9982	0.994	94.6%	0%	17.239	+3.837
EWD	9.669	-0.056	0.9985	0.988	94.0%	0%	21.388	+7.986

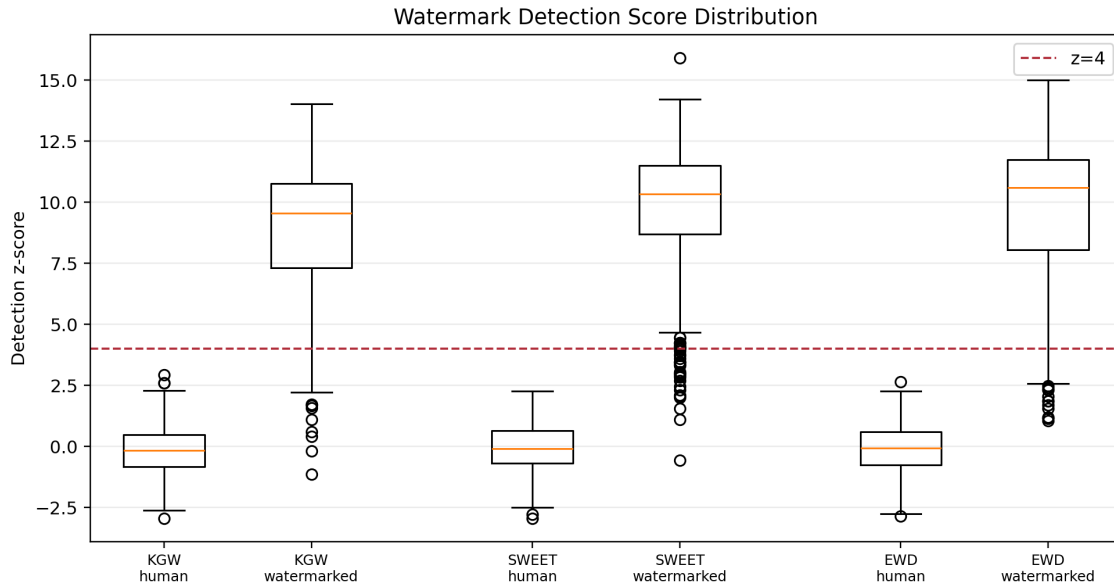


图 1: C4 主实验中 human 与 watermarked z-score 分布

图 1 显示, human completion 的 z-score 集中在 0 附近, 而水印文本整体明显高于 $z = 4$ 阈值。KGW 和 SWEET 均存在少量低 z-score 离群点, 主要来自过短生成或可计分 token 较少的样本; 这说明固定阈值检测仍受文本长度影响。SWEET 的中位数 z-score 更高, 同时 PPL 增量更低, 说明选择性跳过低熵位置在高熵新闻场景中也能降低质量损失。

从误报角度看, 三种方法在 C4 实验中的 human 文本均没有超过 $z > 4$ 阈值, 说明在当前参数下统计检验较保守。需要注意的是, 固定阈值并不能保证所有正样本都被检出: 当生成文本过短、重复、提前结束, 或者可计分 token 较少时, 即使文本由水印模型生成, z-score 也可能低于阈值。因此报告同时使用 AUROC、TPR@5%FPR 和固定阈值检出率, 而不只看单个指标。

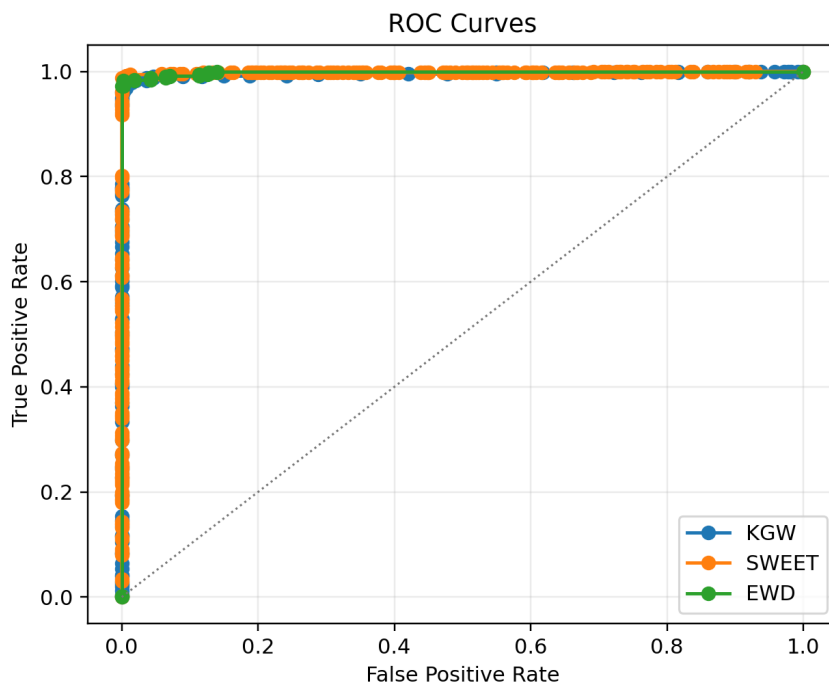


图 2: C4 主实验 ROC 曲线

图 2 中三条 ROC 曲线均贴近左上角，说明排序意义上的检测能力很强。EWD 的 AUROC 最高，但 EWD 复用 KGW 生成文本，因此不能把它理解为独立生成算法优于 KGW；更准确的表述是：在 KGW 生成文本上，EWD 熵加权检测能略微改善检测分数排序。

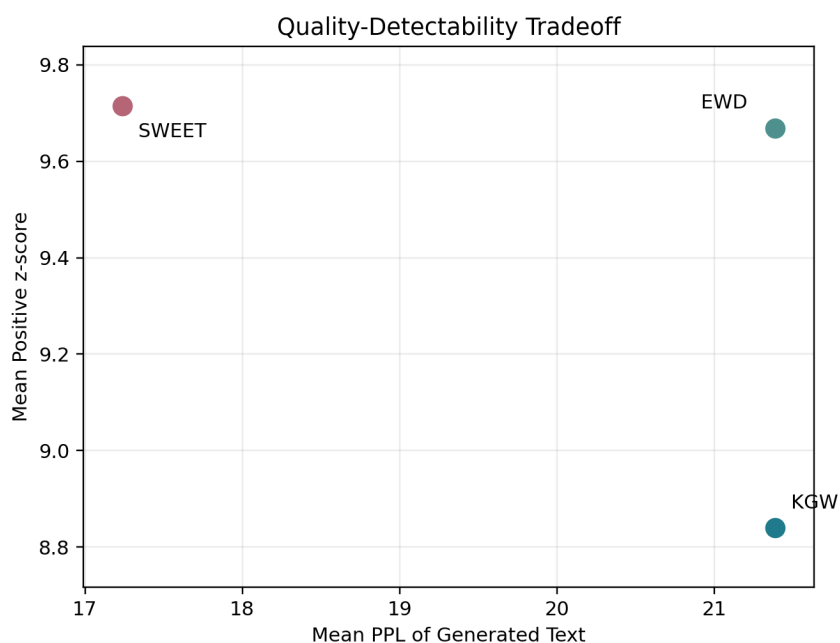


图 3: C4 主实验 PPL 与检测 z-score 权衡

图 3 展示了质量与可检测性的权衡。SWEET 位于更低 PPL、更高 z-score 的区域，说明它在本实验中取得了更好的质量-检测折中。KGW 和 EWD 的 PPL 相同，是因为 EWD 直接复用 KGW 的生成文本。C4 500 中存在一个极端样本只生成了极短 completion，导致 KGW/EWD PPL 达到 1385.43，并拉高平均 PPL；剔除 PPL 大于 100 的极端值后，KGW 平均 PPL 约为 18.655，SWEET 约为 16.965，SWEET 仍然更优。

4.2 低熵代码补充实验

表 2 给出 HumanEval 低熵代码补充实验结果。相比旧合成结构化代码数据，HumanEval 的检测指标不再饱和，更能体现 KGW、SWEET 和 EWD 在真实代码任务中的差异。需要说明的是，本实验仍使用 OPT-1.3B 作为统一生成模型，并以 PPL 作为质量代理指标；它不是 SWEET 论文中使用 StarCoder 并计算 pass@1 的完整代码正确性评测。

HumanEval 的结果更接近低熵场景的预期：所有方法仍然有效，但检测难度明显高于 C4。KGW 的 $z > 4$ 检出率从 C4 的 92.2% 降至 77.4%，说明低熵代码 prompt 下，水印偏置并不总能稳定累积到足够高的 z-score。SWEET 通过跳过低熵位置降低了平均 PPL 增量，并把固定阈值检出率提高到 82.9%。EWD 不改变生成文本，却把检出率提高到 92.1%，说明在低熵检测时，连续熵权重比简单等权统计更能体现水印信号。

表 2: HumanEval 164 条低熵代码补充实验结果

算法	水印 z	human z	AUROC	TPR@5%FPR	$z > 4$ 检出	$z > 4$ 误报	平均 PPL	PPL 增量
KGW	5.820	-1.917	0.9803	0.951	77.4%	0%	12.353	+4.128
SWEET	6.418	-0.417	0.9795	0.957	82.9%	0%	11.248	+3.023
EWD	7.395	-0.483	0.9884	0.988	92.1%	0%	12.353	+4.128

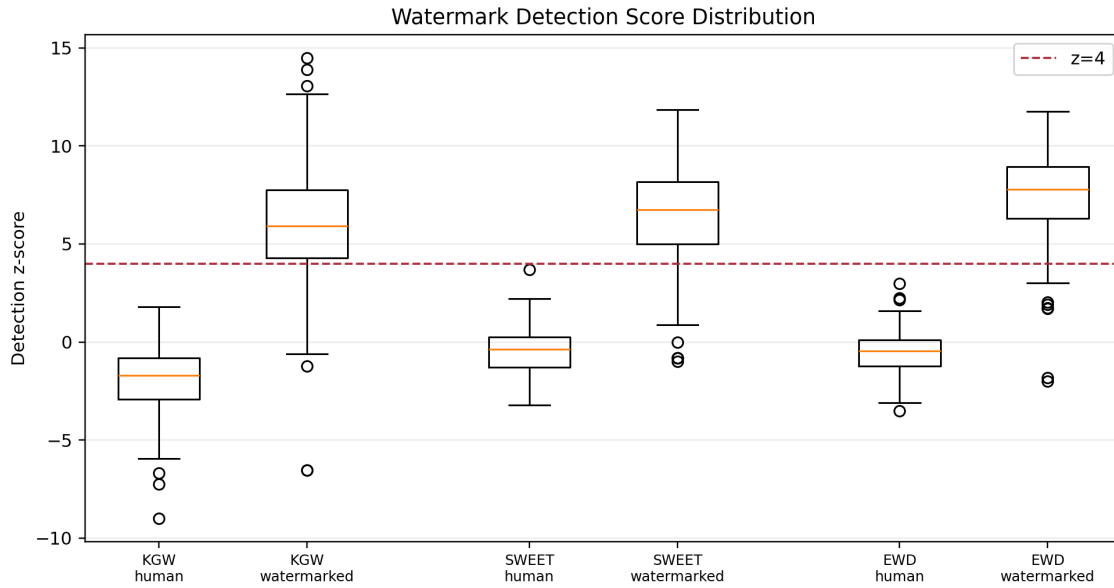


图 4: HumanEval 低熵代码中 human 与 watermarked z-score 分布

图 4 显示，HumanEval 中 watermarked 文本整体高于 human 文本，但三种方法的分离程度已有明显差别。KGW 的水印文本平均 z-score 为 5.820，严格阈值 $z > 4$ 下检出率为 77.4%；SWEET 将平均可计分 token 数从 KGW 的 120.88 降至 61.02，使检出率提高到 82.9%，同时 PPL 增量从 +4.128 降至 +3.023；EWD 复用 KGW 生成文本，但通过熵加权检测把平均 z-score 提高到 7.395，AUROC 提高到 0.9884， $z > 4$ 检出率达到 92.1%。这说明 HumanEval 相比旧合成数据更能体现三种方法的取舍：SWEET 主要改善质量损失，EWD 主要增强检测统计量。

同时也可以看到，SWEET 的 human z-score 均值从 KGW 的 -1.917 上升到 -0.417，这是因为 SWEET 只检测高熵 token，human 文本中可计分位置更少，分数更接近 0。EWD 的 human 均值为 -0.483，仍保持较低误报，并且水印文本均值最高。换言之，EWD 并不是简单抬高所有文本的 z-score，而是更有效地扩大了 watermarked 与 human 的间隔。

4.3 方法优缺点对比

KGW 的优点是结构简单、检测不需要访问模型参数，只要知道密钥和哈希规则即可复原绿名单，因此部署成本低、统计解释清晰。缺点是低熵文本中水印信号可能被固定语法或上下文约束削弱，若提高 δ 又可能损害文本质量。

SWEET 解决的是“哪些 token 值得水印化”的问题：它跳过低熵位置，通常能降低对代码正确性或固定表达的破坏，同时提高有效统计样本中的绿名单比例。当前结果中，SWEET 取得了最低 PPL 增量，说明它在质量保持方面有优势。

EWD 解决的是“检测时怎样使用 token”的问题：它不再二值化地保留或删除 token，而是根据熵连续加权，因此比 KGW 更重视高熵 token，也比 SWEET 更少依赖人工阈

值。不过 EWD 仍需要模型 logits，且本身不是独立生成水印方案。

综合来看，三种方法适合的使用场景并不完全相同。如果系统需要最简单、最容易部署的水印方案，KGW 是清晰的基线；如果生成内容对局部 token 极其敏感，例如代码、公式或固定格式文本，SWEET 更适合用于降低质量损失；如果平台已经能够在检测端访问模型 logits 或代理模型，EWD 可以作为更强的检测统计量叠加在 KGW 生成文本上。本文的实验也支持这一判断：SWEET 的主要收益体现在 PPL，EWD 的主要收益体现在检测分数。

4.4 实验局限性

本文实验仍有若干局限。首先，为了统一框架，HumanEval 实验使用 OPT-1.3B 生成代码，而不是专门的代码模型 StarCoder；因此生成代码的可执行正确性不能直接代表 SWEET 原论文中的 pass@1 结果。其次，PPL 只能作为文本质量的代理指标，尤其在代码任务中，低 PPL 并不必然意味着代码能通过测试。第三，当前实验只覆盖了无攻击场景，没有进一步测试改写、截断、拼接、变量重命名、回译等水印去除攻击。最后，SWEET 的熵阈值和 KGW 的 γ, δ 都会影响质量与检测的平衡，本文固定参数是为了便于比较，但不是最优参数搜索。

这些限制不影响本文的主要对比结论，但说明实验结果应被理解为“统一框架下的复现式比较”，而不是三篇论文完整实验设置的完全再现。若继续扩展，可以优先加入 MBPP 或 HumanEvalPack，并使用代码专用模型计算 pass@1；也可以把检测流程拆成不同攻击强度下的鲁棒性评测，从而更全面地评价水印在真实部署中的可靠性。

5 运行效果与复现

主实验命令如下：

```
python scripts/prepare_assets.py --config configs/opt_full.yaml --hf-mirror --
    skip-models --dataset-samples 500 --dataset-output data/c4_realnewslike_500
    .jsonl
python scripts/run_experiment.py --config configs/opt_cloud_c4_500_ppl.yaml --
    output-dir results/opt_cloud_c4_500
python scripts/plot_results.py --results-dir results/opt_cloud_c4_500
```

低熵代码补充实验命令如下：

```
python scripts/make_humaneval_dataset.py --output data/humaneval_164.jsonl
python scripts/run_experiment.py --config configs/
    opt_cloud_code_low_entropy_ppl.yaml --output-dir results/
    opt_cloud_code_low_entropy
python scripts/plot_results.py --results-dir results/opt_cloud_code_low_entropy
```

当前实现会同时加载 OPT-1.3B 生成模型和 OPT-6.7B PPL/oracle 模型。若云端显存不足，`device_map: auto` 会触发 CPU offload，运行速度会明显下降。后续可改造为两阶段流程：先只加载 OPT-1.3B 完成生成和检测，再释放显存并加载 OPT-6.7B 补算 PPL。

6 总结

本文完成了 KGW、SWEET、EWD 三种 LLM 水印算法的复现式对比。三者的关系可以概括为：KGW 提供基础绿名单偏置框架，SWEET 在生成和检测阶段引入熵阈值选择，EWD 在检测阶段引入连续熵权重。云端正式实验使用 OPT-1.3B/OPT-6.7B，在 C4 RealNewsLike 500 条主实验和 HumanEval 低熵代码补充实验上验证水印检测的有效性。

实验结果表明，在高熵新闻文本中，三种方法均能稳定区分 human 与 watermarked 文本，且 $z > 4$ 阈值下没有误报；在 HumanEval 低熵代码任务中，指标不再饱和，EWD 取得最高 AUROC 和严格阈值检出率，SWEET 则取得最低 PPL 增量。EWD 在本项目中应表述为“KGW 生成 + EWD 熵加权检测”，不能与 KGW、SWEET 完全并列为独立生成算法。

从安全角度看，文本水印的价值不在于给出绝对不可绕过的证明，而在于提供低成本、可量化、可审计的来源证据。未来可以进一步扩展 HumanEval/MBPP 等真实代码数据集，并增加同义改写、回译、截断、拼接攻击等鲁棒性实验。

小组贡献

姓名与贡献待填写。建议提交前补充每位组员在论文阅读、算法实现、实验运行、报告撰写和视频录制中的具体工作。

References

- [1] John Kirchenbauer et al. “A Watermark for Large Language Models”. In: *Proceedings of the 40th International Conference on Machine Learning*. PMLR. 2023, pp. 17061–17084.
- [2] Taehyun Lee et al. “Who Wrote this Code? Watermarking for Code Generation”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024, pp. 4890–4911.
- [3] Yijian Lu et al. “An Entropy-based Text Watermarking Detection Method”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024, pp. 11724–11735.

- [4] Susan Zhang et al. “OPT: Open Pre-trained Transformer Language Models”. In: *arXiv preprint arXiv:2205.01068* (2022).
- [5] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.